InterviewPal — مستند پیاده‌سازی فرانت‌اند

اپلیکیشن تمرین مصاحبهٔ فنی برای برنامه‌نویسان فارسی‌زبان

مورد	مقدار
فناوری	Angular 22 (standalone، zoneless، signals) و TypeScript 6
ریپو	github.com/nasibehash/interviewPal-frontend
اپ زنده	interview-pal-frontend-sable.vercel.app
API	interviewpal-backend.onrender.com (ASP.NET Core، ریپوی interviewPal-backend)
وضعیت	فاز ۲: استفاده از برنامه فقط با حساب کاربری (ثبت‌نام و ورود) ممکن است و پیشرفت روی سرور ذخیره می‌شود

۱. هدف و دامنهٔ محصول

InterviewPal به برنامه‌نویس کمک می‌کند برای مصاحبهٔ فنی تمرین کند. محتوا فارسی است و اصطلاح‌های فنی و کدها انگلیسی می‌مانند. فرانت‌اند دو بخش اصلی دارد:

- تمرین مصاحبه: ۳۰۰ سؤال در شش فناوری (Next.js، React، TypeScript، JavaScript، Angular و .NET) در سه سطح جونیور، مید و سنیور و سه حالت تمرین: یادگیری، مصاحبه (با تایمر) و فلش‌کارت.
- الگوریتم و الگوهای طراحی: ۲۴ درس (۱۴ الگوریتم و ۱۰ الگوی طراحی) که هر کدام مثال واقعی دارد و کدش به زبان یا فریم‌ورک انتخابی کاربر نمایش داده می‌شود، همراه با تمرین.
- راهنمای Angular برای سنیور: ۲۱ مبحث کوتاه در ۶ گروه (Signals، RxJS، قالب و رندر، معماری، داده و فرم، کیفیت) که هر کدام ایدهٔ اصلی، یک مثال کامل و نکته‌های سنیور دارند، به‌علاوهٔ بخش جدای «سؤال‌های مصاحبهٔ سنیور» با ۷ مبحث سؤال و جواب (Signals، change detection و DI، RxJS و معماری، Router و HTTP، امنیت، تست و SSR، سؤال‌های تجربه‌محور).
- پیشرفت من: تاریخچهٔ تمرین‌ها، میانگین نمره، روزهای پشت‌سرهم و سؤال‌های ضعیف.
- حساب کاربری: ثبت‌نام و ورود با ایمیل و رمز عبور. با ورود، تاریخچه و پیشرفت درس‌ها روی سرور ذخیره می‌شود و در هر دستگاه در دسترس است؛ داده‌های قبلی همان مرورگر (از نسخه‌های بدون حساب) هنگام اولین ورود منتقل می‌شوند. بدون ورود فقط صفحه‌های ورود و ثبت‌نام باز می‌شوند.
- ریسپانسیو: طرح از عرض ۳۶۰ پیکسل (موبایل) تا دسکتاپ بدون اسکرول افقی کار می‌کند؛ منو در موبایل ردیف جدا و قابل‌اسکرول دارد.

۲. پشتهٔ فناوری و تصمیم‌های اصلی

حوزه	انتخاب	دلیل
فریم‌ورک	Angular 22، کامپوننت‌های standalone	آخرین نسخه؛ بدون NgModule و با بارگذاری تنبل ساده

حوزه	انتخاب	دلیل
تشخیص تغییر	Zoneless و `OnPush` (هر دو پیش‌فرض Angular 22)	بدون zone.js؛ همهٔ state با signal است و فقط بخش تغییرکرده بررسی می‌شود
state	signal و computed در سرویس‌های root	بدون کتابخانهٔ state جدا؛ جریان داده ساده و قابل‌تست
داده از API	`httpResource` (داخل `ApiClient`) برای خواندن، `firstValueFrom` برای نوشتن	خواندن‌ها reactiveاند و با تغییر signal دوباره درخواست می‌شوند
فرم‌ها	Signal Forms (`@angular/forms/signals`)	مدل فرم یک signal ساده است؛ بدون `FormsModule` و `ngModel`
قالب	کنترل‌فلو `@if` / `@for` / `@empty`	سینتکس جدید Angular، بدون `*ngIf`
زبان و جهت	فارسی، `dir="rtl"` و خصوصیت‌های منطقی CSS	یک پایه برای RTL؛ کدها جداگانه LTR نمایش داده می‌شوند
فونت	Vazirmatn (بسته `@fontsource-variable/vazirmatn`)	بدون درخواست به CDN بیرونی
Markdown	`marked` و `DOMPurify` و `highlight.js`	محتوای درس‌ها Markdown است؛ بعد از تبدیل پاک‌سازی (sanitize) می‌شود
احراز هویت	JWT کوتاه‌عمر در حافظه و refresh token در کوکی httpOnly	توکن دسترسی هیچ‌جا ذخیره نمی‌شود و جاوااسکریپت به refresh token دسترسی ندارد (جلوگیری از سرقت با XSS)
تست	Vitest از طریق `ng test`	اجرای سریع و ساده؛ ۹۳ تست واحد
استایل	SCSS ساده با متغیرهای CSS و تم روشن/تیره	بدون کتابخانهٔ UI؛ حجم کم

ذخیره‌سازی بدون حساب و با حساب

کاربر مهمان: تاریخچه و پیشرفت در `localStorage` و تمرین نیمه‌کاره در `sessionStorage` می‌ماند. همهٔ خواندن و نوشتن‌ها از یک لایهٔ امن (safe-storage) می‌گذرند تا در حالت مرور خصوصی یا ذخیره‌سازی مسدود، برنامه نشکند. کاربر واردشده: `ProgressStore` و `LessonProgress` همان API قبلی را نگه می‌دارند ولی منبع داده سرور است؛ پس صفحه‌ها تغییری نکردند (بخش ۵.۴).

۳. ساختار پروژه

```
src/app/
  app.ts / app.html / app.scss   shell: header, nav, router-outlet
  app.config.ts                  providers: http + interceptors, router features, session
restore
  app.routes.ts                  lazy routes and guards
  core/                          logic and data, no UI
    models.ts                    TypeScript types of the API contract
    api-client.ts                the only place that talks HTTP (resources + calls)
    guides.ts                    GuideCatalog: index of the Angular guides (httpResource)
    api-error.ts                 interceptor: HTTP failure -> ApiError (Persian message)
    auth-store.ts                user, access token (memory), login/register/logout/restore
    auth-interceptor.ts          adds the bearer token, refreshes once on 401
    auth-guards.ts               authGuard / guestGuard
    practice-session.ts          state of the practice in progress
    progress-store.ts            history and stats (localStorage or server)
    lesson-progress.ts           progress of lesson exercises (localStorage or server)
    preferred-technology.ts      the language the learner picked
    self-assessment.ts           "I knew it" -> a gradable answer
    safe-storage.ts              storage reads/writes that never throw
  shared/                        shared components (ts + html + scss each)
    markdown.ts, markdown.pipe.ts, question-view, answer-panel,
    report-dialog, code-block, field-errors
  features/                      pages, each one lazy loaded
    setup/  practice/  result/  history/  lessons/  guides/  auth/  account/
public/content/angular/          the guides: index.json + one Markdown file per topic
scripts/check-guides.mjs         checks that index.json and the files agree (runs before npm
test)
docs/
  frontend.md                    this document (source)
  InterviewPal-Frontend.pdf      the same document as PDF
```

در این ساختار، هر پوشه یک مسئولیت دارد: `core` منطق و داده، `shared` اجزای تکراری و `features` صفحه‌ها.

قرارداد نام‌گذاری فایل‌ها

از Angular 20 به بعد راهنمای رسمی سبک (style guide) و دستور `ng generate` پسوند `.component` / `.service` را حذف کرده‌اند. به‌جای `lesson.component.ts` نام فایل `lesson.ts` و نام کلاس `Lesson` است (نه `LessonComponent`). این پروژه همین قرارداد را دنبال می‌کند:

- هر کامپوننت سه فایل جدا دارد: `name.ts` ، `name.html` و `name.scss` (با `templateUrl` و `styleUrl`). قالب و استایل inline فقط در تست‌ها استفاده می‌شود.
- نام صفحه‌ها پسوند `-page` دارد (`lesson-page.ts`) تا از کامپوننت‌های کوچک‌تر در پوشهٔ `shared` جدا شوند؛ این یک انتخاب پروژه است، نه الزام Angular.
- سرویس‌ها بدون پسوند و با نام معنادار: `ApiClient` ، `ProgressStore` .

قاعدهٔ اصلی: `core` به هیچ کامپوننتی وابسته نیست و `features` فقط از `core` و `shared` استفاده می‌کنند. هر صفحه با `loadComponent` بارگذاری می‌شود و bundle اولیه کوچک می‌ماند (حدود ۹۲ کیلوبایت فشرده).

۴. مسیریابی

مسیر	صفحه	توضیح
`/`	شروع تمرین	انتخاب تکنولوژی، سطح، تعداد (۵ تا ۱۰۰) و حالت؛ همهٔ مسیرها به‌جز `/login` و `/register` پشت `authGuard` هستند
`/practice`	تمرین	guard: فقط وقتی تمرین فعال وجود دارد
`/result`	نتیجه	guard: فقط بعد از تمام شدن و ارزیابی
`/lessons`	فهرست درس‌ها	تب الگوریتم‌ها و الگوهای طراحی، فیلتر سطح
`/lessons/:id`	درس	پارامتر `id` با `withComponentInputBinding` مستقیم به `input()` می‌رسد
`/guides`	راهنمای Angular	فهرست مبحث‌ها به‌تفکیک گروه، همراه جست‌وجوی debounceشده
`/guides/:slug`	مبحث	متن Markdown مبحث و پیوند قبلی/بعدی؛ `slug` با `withComponentInputBinding` به `input()` می‌رسد
`/history`	پیشرفت من	آمار و تاریخچه
`/login`	ورود	`guestGuard`: کاربر واردشده به `/` یا مسیر `redirect` می‌رود؛ مهمان بعد از ورود به همان صفحهٔ خواسته‌شده برمی‌گردد
`/register`	ثبت‌نام	فقط مهمان؛ بعد از موفقیت مستقیم وارد می‌شود
`/account`	حساب من	guard: فقط کاربر واردشده؛ مهمان به `/login?redirect=/account` می‌رود
`**`	—	بازگشت به صفحهٔ اصلی

guardها تابعی (`CanActivateFn`) هستند. guardهای تمرین فقط وضعیت `PracticeSessionStore` را می‌خوانند: اگر کاربر مستقیم `/practice` را باز کند و تمرینی نداشته باشد، به `/` هدایت می‌شود. `authGuard` و `guestGuard` اول منتظر `auth.ready` (پایان بازیابی نشست) می‌مانند تا با refresh صفحه کاربر واردشده به‌اشتباه به `/login` نرود.

۵. لایهٔ core

۵.۱ مدل‌ها و ApiClient

`models.ts` آیینهٔ قرارداد JSON بک‌اند است (camelCase و enumها به‌صورت رشته). سؤال‌ها بدون جواب می‌آیند و جواب فقط بعد از پاسخ‌دادن از API گرفته می‌شود، پس جواب در payload اولیه نیست. `ApiClient` تنها جایی است که `HttpClient` را می‌شناسد و آدرس پایه از توکن `API_BASE_URL` (پیش‌فرض `/api`) می‌آید. دو دسته متد دارد:

- **خواندن به‌صورت resource:** `technologiesResource()`، `lessonsResource()` و `lessonResource(id, technology)` یک `httpResource` برمی‌گردانند. صفحه‌ها آن را در field initializer می‌سازند و فقط `value()`،

`isLoading()` ، `error()` و `reload()` را می‌بینند؛ هیچ صفحه‌ای URL نمی‌سازد. `lessonResource` دو تابع (برای `id` و زبان) می‌گیرد و با تغییر هرکدام خودش دوباره درخواست می‌زند.

- **نوشتن و بقیه به‌صورت Observable:** ارزیابی پاسخ، ساخت تمرین و گزارش، که در stateها با `firstValueFrom` مصرف می‌شوند.

`apiErrorInterceptor` (فایل `api-error.ts`) هر خطای HTTP را به `ApiError` با پیام فارسی تبدیل می‌کند: ۰ (شبکه)، ۴۰۰ (متن ProblemDetails سرور)، ۴۰۴، ۴۲۹ و ۵xx. بنابراین storeها و صفحه‌ها هیچ‌وقت `HttpErrorResponse` خام نمی‌بینند.

۵.۲ PracticeSessionStore

قلب بخش تمرین است و کل state را با signal نگه می‌دارد: `session` ، `answers` ، `revealed` ، `correctness` ، `notes` ، `index` ، `startedAt` ، `phase` ، `evaluation` ، `busy` و `error` . فاز تمرین یکی از مقدارهای زیر است:

```
idle -> practicing -> (self-assessment) -> submitting -> finished
              ^                                    |
              +------ evaluation failed ---------------+
```

- حالت یادگیری: با انتخاب هر گزینه، پاسخ فوراً به API (`check`) فرستاده می‌شود و درست/غلط و توضیح نشان داده می‌شود؛ پاسخ قفل می‌شود.
- حالت مصاحبه: پاسخ‌ها فقط در حافظه جمع می‌شوند و در پایان با یک درخواست (`evaluate`) ارزیابی می‌شوند. سؤال‌های تشریحی که کاربر خودارزیابی نکرده، قبل از ارسال وارد فاز `self-assessment` می‌شوند.
- حالت فلش‌کارت: جواب نمایش داده می‌شود و کاربر خودش «بلد بودم / بلد نبودم» می‌زند.
- سؤال بی‌جواب در انتها غلط حساب می‌شود (`completeAnswers`).
- تمرین نیمه‌کاره در `sessionStorage` ذخیره می‌شود و با refresh از همان سؤال ادامه پیدا می‌کند (با `effect`).
- اگر ارزیابی با خطا روبه‌رو شود، فاز به `practicing` برمی‌گردد تا کاربر دوباره تلاش کند.

تابع `answerFromSelfAssessment` نتیجهٔ خودارزیابی را به پاسخی تبدیل می‌کند که API بفهمد: برای سؤال تشریحی `knewIt` و برای سؤال گزینه‌ای، گزینهٔ درست (اگر کاربر بلد بوده) یا یک گزینهٔ غلط.

۵.۳ ProgressStore و دیگر سرویس‌ها

سرویس	کلید storage	کار
ProgressStore	`interviewpal.history.v1` و `interviewpal.question-stats.v1`	حداکثر ۱۰۰ تمرین اخیر؛ آمار هر سؤال؛ میانگین، streak و سؤال‌های ضعیف (computed)
PracticeSessionStore	`interviewpal.active-session.v1` (sessionStorage)	تمرین در حال انجام
LessonProgress	`interviewpal.lesson-progress.v2`	پاسخ آخر هر تمرین درس؛ درس وقتی کامل است که همهٔ تمرین‌ها پاسخ داده و آخرین پاسخ‌ها درست باشند

سرویس	کلید storage	کار
PreferredTechnology	`interviewpal.technology.v1`	زبان انتخابی؛ مقدار نامعتبر به `javascript` برمی‌گردد

نسخه (`.v1` ، `.v2`) در نام کلیدها گذاشته شده تا اگر ساختار داده عوض شد، داده‌های قدیمی بی‌صدا نادیده گرفته شوند و برنامه نشکند.

۵.۴ حساب کاربری و همگام‌سازی

AuthStore وضعیت کاربر را با signal نگه می‌دارد: `user` ، `isLoggedIn` و `ready` . توکن دسترسی (JWT، ۱۵ دقیقه) فقط در حافظهٔ همین سرویس است و در storage نوشته نمی‌شود. refresh token را سرور در کوکی `httpOnly` و `SameSite=Strict` با مسیر `/api/auth` می‌گذارد و جاوااسکریپت آن را نمی‌بیند.

- **بازیابی نشست:** `provideAppInitializer` متد `restore()` را صدا می‌زند. فقط اگر قبلاً نشانهٔ ورود (یک پرچم غیرحساس در storage) ثبت شده باشد، `POST /api/auth/refresh` فرستاده می‌شود؛ مهمان‌ها درخواست اضافه نمی‌فرستند.
- **`authInterceptor`:** روی درخواست‌های `/api` ، هدر `Authorization: Bearer` اضافه می‌کند. با پاسخ ۴۰۱ یک بار refresh می‌زند (چند درخواست هم‌زمان یک refresh مشترک دارند) و درخواست را تکرار می‌کند؛ اگر refresh رد شد، کاربر خارج می‌شود. ترتیب interceptorها `[apiErrorInterceptor, authInterceptor]` است تا خطای ۴۰۱ خام به interceptor احراز هویت برسد.
- **ورود و ثبت‌نام:** فرم‌ها Signal Formsاند. در ثبت‌نام، تکرار رمز با `validate` و `valueOf` به فیلد رمز مقایسه می‌شود (اعتبارسنجی بین‌فیلدی). خطاهای فیلد را کامپوننت مشترک `FieldErrors` نشان می‌دهد.
- **حساب من:** تغییر رمز (سرور همهٔ نشست‌ها را می‌بندد و کاربر با رمز جدید دوباره وارد می‌شود)، حذف حساب با تأیید رمز و حذف پیشرفت ذخیره‌شده روی سرور.

همگام‌سازی پیشرفت: `ProgressStore` و `LessonProgress` با یک `effect` روی `auth.user()` بین دو منبع جابه‌جا می‌شوند.

وضعیت	منبع داده	رفتار
مهمان	—	به صفحه‌ای جز ورود و ثبت‌نام دسترسی ندارد؛ `localStorage` فقط بقایای نسخه‌های قبلی را نگه می‌دارد
ورود	سرور (`/api/me/...`)	با ورود، تاریخچه و پیشرفت درس‌ها بارگذاری می‌شود؛ نتیجهٔ تمرین بعدی را خود سرور هنگام ارزیابی ثبت می‌کند
اولین ورود	`POST /api/me/progress/import`	داده‌های محلی یک‌بار منتقل می‌شوند و بعد از موفقیت از مرورگر پاک می‌شوند
خروج	`localStorage`	دوباره حالت مهمان؛ داده‌های حساب قبلی در مرورگر نمی‌ماند، چون بعد از ورود از سرور خوانده می‌شود و نسخهٔ محلی بعد از انتقال پاک شده است

بک‌اند هم همین قاعده را اعمال می‌کند: مسیرهای سؤال، درس و تمرین بدون توکن `401` می‌دهند، پس دور زدن رابط کاربری ممکن نیست. اگر نشست وسط کار تمام شود (خروج، refresh ردشده، حذف حساب)، `App` کاربر را به `/login` می‌برد.

۶. صفحه‌ها

۶.۱ شروع تمرین (setup)

فهرست تکنولوژی‌ها با `httpResource` از `/technologies` می‌آید و حالت‌های loading و خطا (با دکمهٔ «تلاش دوباره») دارد. انتخاب تکنولوژی و سطح چندگانه است و خالی بودن یعنی «همه». تعداد سؤال با اسلایدر ۵ تا ۱۰۰ و زمان تقریبی (حدود ۸۰ ثانیه برای هر سؤال) نشان داده می‌شود.

۶.۲ تمرین (practice)

- نوار پیشرفت و شمارندهٔ «سؤال n از N»؛ دکمه‌های قبلی/بعدی و «پایان تمرین» در سؤال آخر.
- در حالت مصاحبه تایمر معکوس از `timeLimitSeconds` نشان داده می‌شود؛ زیر یک دقیقه قرمز می‌شود و با صفر شدن، تمرین خودکار ارسال می‌شود.
- گزینه‌ها بعد از پاسخ به رنگ درست (سبز) و غلط (قرمز) درمی‌آیند و پاسخ کامل (جواب کوتاه، توضیح، اشتباه رایج و سؤال بعدی مصاحبه‌کننده) نمایش داده می‌شود.
- گزارش مشکل یک سؤال (`report-dialog`) بعد از دیدن جواب در دسترس است.

۶.۳ نتیجه (result)

درصد و تعداد پاسخ درست، نمودار میله‌ای به تفکیک تکنولوژی و سطح، موضوع‌های ضعیف (tag) و مرور همهٔ سؤال‌ها با توضیح. دکمهٔ «تمرین دوباره با سؤال‌های ضعیف» با `startFromQuestions` یک تمرین یادگیری از همان سؤال‌ها می‌سازد.

۶.۴ پیشرفت من (history)

تعداد تمرین، میانگین نمره، روزهای پشت‌سرهم، دقت هر تکنولوژی، فهرست سؤال‌های ضعیف و ۱۵ تمرین آخر. دکمهٔ پاک‌کردن سابقه بعد از تأیید کاربر همهٔ داده‌ها را حذف می‌کند.

۶.۵ درس‌ها (lessons)

- فهرست: انتخاب زبان در بالای صفحه، دو تب (الگوریتم‌ها / الگوهای طراحی)، فیلتر سطح، گروه‌بندی بر اساس دسته و نشان «✓ کامل» روی درس‌های تمام‌شده.
- جست‌وجو: یک Signal Form با `debounce` ۲۵۰ میلی‌ثانیه؛ در عنوان، خلاصه و تگ‌ها می‌گردد و حروف عربی و فارسی («ك»/«ک» و «ي»/«ی») را یکی می‌گیرد.
- صفحهٔ درس: مثال واقعی، توضیح، پیچیدگی (برای الگوریتم‌ها)، کد به زبان انتخابی با دکمهٔ کپی، شرح کد، «کِی استفاده کنم / کِی نه»، اشتباه رایج و ۴ تمرین.
- عوض کردن زبان در وسط درس، درس را با `technology` جدید دوباره از API می‌گیرد؛ چون `httpResource` به signal زبان وابسته است.
- بخش تمرین‌ها داخل `@defer (on viewport)` است: کد آن (کامپوننت `LessonExerciseView`) در یک chunk جدا بارگذاری می‌شود و فقط وقتی کاربر به پایین صفحه برسد.

- هر تمرین جدا با API بررسی می‌شود (جواب درست در payload درس نیست) و نتیجه در `LessonProgress` ثبت می‌شود.

۶.۶ راهنمای Angular (guides)

- **محتوا فایل ایستا است:** `public/content/angular/index.json` گروه‌ها و مبحث‌ها (slug، عنوان، خلاصه، دقیقه) را نگه می‌دارد و هر مبحث یک فایل `<slug>.md` است. بدون بک‌اند و بدون build جدا سرو می‌شود؛ افزودن مبحث یعنی یک فایل Markdown و یک ردیف در index.
- **`GuideCatalog` :** index را با `httpResource` می‌خواند. چون خواندن `value()` یک resource خطادار خطا پرتاب می‌کند، یک `computed` به نام `data` در حالت خطا و بارگذاری index خالی می‌دهد.
- **`GuidePage` :** فقط slugهای موجود در index را درخواست می‌کند، چون Vercel مسیر ناشناخته را به `index.html` برمی‌گرداند و بدون این بررسی HTML به‌عنوان Markdown نمایش داده می‌شد. متن با `httpResource.text` گرفته و با همان `markdown` pipe (پاک‌سازی با DOMPurify و رنگ‌آمیزی) رندر می‌شود.
- **بخش مصاحبه:** گروه `interview` در همان index است؛ هر مبحثش مجموعه‌ای از سؤال با «جواب» کوتاه (قابل‌گفتن در مصاحبه) و در صورت لزوم مثال کد است. مبحث تجربه‌محور به‌جای جواب آماده، قالب پاسخ (زمینه، تشخیص، اقدام، نتیجه، درس) و اسکلت‌هایی با جای خالی می‌دهد، چون جواب باید از تجربهٔ خود فرد باشد.
- **درستی مثال‌ها:** همهٔ کدهای TypeScript با کامپایلر Angular 22 در حالت `strict` و `strictTemplates` کامپایل شدند، مثال‌های RxJS اجرا شدند و خروجی کامنت‌ها با اجرای واقعی تطبیق داده شد، مثال تست با Vitest اجرا شد، و مثال refresh هم‌زمان توکن با سه درخواست ۴۰۱ همزمان تست شد (فقط یک refresh و سه تکرار).
- **`scripts/check-guides.mjs` :** پیش از `npm test` اجرا می‌شود و تطابق index و فایل‌ها، یکتا بودن slug، گروه معتبر، تعادل fenceها، زبان‌های قابل‌رنگ‌آمیزی و نبود `<script>` را بررسی می‌کند.

۷. کامپوننت‌های مشترک

کامپوننت	کار
markdown.ts و MarkdownPipe	تبدیل Markdown به HTML، رنگ‌آمیزی کد با `highlight.js` (فقط زبان‌های لازم ثبت شده‌اند: JavaScript، TypeScript، HTML، C#، JSON، bash، CSS)، پاک‌سازی با `DOMPurify` و فقط بعد از آن `bypassSecurityTrustHtml`
QuestionView	متن سؤال، سطح (رنگی)، تگ‌ها و قطعهٔ کد (همیشه `dir="ltr"`)
AnswerPanel	جواب کوتاه، توضیح کامل، اشتباه رایج و سؤال بعدی
ReportDialog	گزارش جواب غلط، قدیمی، مبهم یا غلط نگارشی؛ با Signal Form، `required` ، `maxLength(500)` و `<form [formRoot]>`
CodeBlock	نمایش کد با زبان و دکمهٔ «کپی کد» (clipboard)؛ خروجی `highlight.js` خودش کد را escape می‌کند

امنیت

همهٔ متن‌های Markdown (سؤال، توضیح، درس) قبل از رسیدن به DOM با DOMPurify پاک‌سازی می‌شوند. اسکریپت و `onerror` حذف می‌شود و این رفتار تست واحد دارد. هیچ `innerHTML` دیگری بدون این مسیر استفاده نشده است.

۸. ویژگی‌های Angular 22 که در پروژه استفاده شده

ویژگی	کجا	چرا
Zoneless (پیش‌فرض)	`bootstrapApplication` بدون zone.js	رندر فقط با تغییر signal؛ بدون patch کردن APIهای مرورگر
`OnPush` پیش‌فرض	همهٔ کامپوننت‌ها (بدون تنظیم صریح)	در Angular 22 پیش‌فرض است؛ همهٔ state با signal است، پس سازگار است
`httpResource` (پایدار از 22.0)	`ApiClient.technologiesResource()`، `lessonsResource()`، `lessonResource()`	خواندن reactive: با تغییر `id` یا زبان خودش دوباره درخواست می‌زند و `reload` / `error` / `isLoading` می‌دهد؛ `defaultValue: []` الگوی `?? []` را از قالب‌ها حذف کرد
Signal Forms (پایدار از 22.0)	`ReportDialog` و جست‌وجوی `LessonsPage`	`form()`، `[formField]`، `<form [formRoot]>` با `submission`، `required`، `maxLength` و `debounce`؛ مدل فرم یک `signal` ساده است
`linkedSignal`	`LessonExerciseView`	state قابل‌نوشتن که با عوض شدن `exercise` ورودی خودش بازنشانی می‌شود
`@defer (on viewport)`	تمرین‌های `LessonPage`	کد تمرین‌ها در یک chunk جدا و فقط با رسیدن به پایین صفحه بارگذاری می‌شود
`@let`	`LessonPage`	متغیر محلی در قالب، به‌جای تکرار `technology.slug()`
کنترل‌فلو جدید	همهٔ قالب‌ها	`@if`، `@for` با `track`، `@empty`
`input()` / `input.required()`	کامپوننت‌ها و `LessonPage`	ورودی‌های signal؛ با `withComponentInputBinding` پارامتر `:id` مسیر مستقیم به `input` می‌رسد
`computed` و `effect`	storeها و صفحه‌ها	مشتق‌های خالص با `computed`؛ `effect` فقط برای هماهنگ‌سازی با `sessionStorage` و تایمر

ویژگی	کجا	چرا
Functional interceptor	`apiErrorInterceptor` و `authInterceptor` با `withInterceptors`	تبدیل خطاها به `ApiError` و افزودن/تازه‌سازی توکن در یک نقطه
Router	`withViewTransitions`، `withInMemoryScrolling`، guardهای تابعی، `loadComponent`	انتقال انیمیشنی بین صفحه‌ها، برگرداندن اسکرول و bundle اولیهٔ کوچک
`provideAppInitializer`	`app.config.ts`	بازیابی نشست قبل از اولین رندر؛ guardها با `auth.ready` منتظر همین کار می‌مانند
Signal Forms: `validate` و `valueOf`	`RegisterPage`، `AccountPage`	اعتبارسنجی بین‌فیلدی (تکرار رمز) بدون کد دستی؛ پیام‌ها را از `FieldErrors` یا `field.errors()` می‌خواند
`inject()`	همه‌جا	بدون constructor injection؛ کار با field initializer و تابع‌های کمکی ساده می‌شود
Vitest با `ng test`	همهٔ تست‌ها	builder `@angular/build:unit-test`؛ بدون Karma و Jasmine

بعضی از این‌ها، مثل `httpResource` و Signal Forms، در نسخه‌های قبلی آزمایشی بودند. در Angular 22 هر دو در تایپ‌های خود فریم‌ورک با `@publicApi 22.0` (یعنی API عمومی و پایدار) علامت خورده‌اند.

۹. استایل، RTL و دسترس‌پذیری

- متغیرهای CSS در `:root` و تم تیره با `prefers-color-scheme`؛ رنگ اصلی بنفش (`#4F46E5`) و رنگ‌های جدا برای موفقیت، خطا و هشدار.
- خصوصیت‌های منطقی (`margin-inline`، `inline-size`، `inset-block-start`) به‌جای `left/right` تا جهت RTL خودکار درست شود.
- کد و عبارت‌های انگلیسی `dir="ltr"` و `unicode-bidi: plaintext` دارند تا در متن فارسی به‌هم نریزند.
- دکمه‌های انتخابی `aria-pressed`، تب‌ها `role="tab"` و `aria-selected`، تایمر `role="timer"`، خطاها `role="alert"` و نوار پیشرفت `role="progressbar"` دارند. focus قابل‌مشاهده است.
- `prefers-reduced-motion` انیمیشن‌ها را خاموش می‌کند و طرح روی عرض موبایل بدون اسکرول افقی کار می‌کند.

۱۰. قرارداد API

متد	مسیر	استفاده
GET	`/api/technologies`	فهرست تکنولوژی‌ها و تعداد سؤال هر سطح

متد	مسیر	استفاده
POST	`/api/practice/sessions`	ساخت تمرین (تکنولوژی‌ها، سطح‌ها، تعداد، حالت)
POST	`/api/practice/questions/{id}/check`	ارزیابی یک پاسخ (یادگیری)
POST	`/api/practice/evaluate`	ارزیابی کل تمرین (مصاحبه)
GET	`/api/questions/{id}`	یک سؤال با جواب (برای تمرین دوباره و فلش‌کارت)
POST	`/api/questions/{id}/reports`	گزارش مشکل سؤال
GET	`/api/lessons?kind=`	فهرست درس‌ها
GET	`/api/lessons/{id}?technology=`	درس با کد زبان انتخابی
POST	`/api/lessons/{id}/exercises/{exerciseId}/check`	بررسی یک تمرین درس (با ورود، نتیجه ذخیره می‌شود)
POST	`/api/auth/register`	ثبت‌نام؛ توکن دسترسی در بدنه و refresh token در کوکی
POST	`/api/auth/login`	ورود (قفل موقت بعد از چند رمز اشتباه و محدودیت تعداد درخواست)
POST	`/api/auth/refresh`	توکن دسترسی جدید و چرخش refresh token
POST	`/api/auth/logout`	پایان نشست و پاک‌کردن کوکی
GET	`/api/auth/me`	اطلاعات کاربر فعلی
POST	`/api/auth/change-password`	تغییر رمز؛ همهٔ نشست‌ها بسته می‌شوند
POST	`/api/auth/delete-account`	حذف حساب و دادهٔ آن با تأیید رمز
GET	`/api/me/progress`	تاریخچه، آمار سؤال‌ها و پیشرفت درس‌های کاربر
POST	`/api/me/progress/import`	انتقال یک‌بارهٔ دادهٔ محلی به حساب
DELETE	`/api/me/progress`	پاک‌کردن پیشرفت ذخیره‌شده

مسیرهای `/api/me/*` و تغییر رمز و حذف حساب نیاز به هدر `Authorization` دارند. کوکی refresh فقط به `/api/auth` فرستاده می‌شود و چون Vercel (و nginx و پراکسی توسعه) `/api` را روی همان origin پراکسی می‌کنند، مرورگر آن را همان‌سایتی می‌بیند و CORS لازم نیست.

خطاها به‌صورت ProblemDetails می‌آیند. `PracticeSessionStore` پیام فارسی عمومی نشان می‌دهد و صفحه‌ها برای خطای بارگذاری دکمهٔ «تلاش دوباره» دارند.

۱۱. تست

۹۳ تست واحد با Vitest (`npm test`):

- ApiClient: مسیر و بدنهٔ هر درخواست با `HttpTestingController` .
- PracticeSessionStore: شروع تمرین، ارزیابی فوری در یادگیری، نبودن درخواست در مصاحبه، فاز خودارزیابی، ارسال نهایی، بازگشت بعد از خطا و تمرین خالی.
- LessonProgress، ProgressStore و PreferredTechnology: ثبت، ماندگاری، پاک‌کردن، streak و مقدار نامعتبر storage.
- GuidesPage و GuidePage: گروه‌بندی و جست‌وجو، خطای بارگذاری فهرست، رندر Markdown با پاک‌سازی script، مبحث قبلی/بعدی و درخواست‌ندادن برای slug ناموجود.
- `apiErrorInterceptor` : تبدیل خطاهای HTTP به `ApiError` با پیام فارسی.
- AuthStore: ورود، ثبت‌نام، بازیابی نشست (با و بدون نشانه)، خروج و تغییر رمز.
- `authInterceptor` : افزودن توکن، یک refresh مشترک برای چند ۴۰۱ هم‌زمان، تکرار درخواست و خروج بعد از رد شدن refresh.
- `authGuard` و `guestGuard` : انتظار برای بازیابی نشست و هدایت با `redirect` .
- همگام‌سازی پیشرفت: جابه‌جایی منبع داده با ورود/خروج و انتقال یک‌بارهٔ داده‌های محلی.
- صفحه‌های ورود، ثبت‌نام و حساب (Signal Forms، اعتبارسنجی بین‌فیلدی، خطاهای سرور) و پیام‌های فارسی خطای احراز هویت.
- `ReportDialog` (Signal Form): ارسال دلیل و پیام و جلوگیری از ارسال پیام بیش از ۵۰۰ کاراکتر.
- `LessonExerciseView` : بازنشانی پاسخ قبلی با `linkedSignal` وقتی تمرین عوض می‌شود.
- markdown: رنگ‌آمیزی، حذف اسکریپت و `onerror` ، escape زبان‌های ناشناخته.
- LessonsPage، SetupPage و LessonPage: لیست، فیلتر، جست‌وجوی debounceشده با حروف عربی/فارسی، انتخاب زبان، گرفتن درس با زبان جدید و بررسی تمرین. بلوک `@defer` در تست با `DeferBlockBehavior.Manual` دستی رندر می‌شود، چون jsdom از `IntersectionObserver` پشتیبانی نمی‌کند.

علاوه بر آن، کل جریان یادگیری و جریان حساب (ثبت‌نام با انتقال تاریخچه، بازیابی بعد از refresh صفحه، تغییر رمز، حذف حساب و بررسی کوکی httpOnly) یک بار در مرورگر واقعی (Chromium) روی بک‌اند واقعی اجرا و تأیید شده است.

۱۲. اجرا و استقرار

۱۲.۱ توسعه

```
npm install
npm start      # http://localhost:4200  (/api به http://localhost:5161 پراکسی می‌شود)
npm test
npm run build  # خروجی: dist/interviewpal/browser
```

Angular CLI 22 به Node نسخهٔ ۲۲٫۲۲ یا بالاتر نیاز دارد (Node 24 پیشنهاد می‌شود). بک‌اند باید روی `http://localhost:5161` در حال اجرا باشد (`proxy.conf.json`).

۱۲.۲ Docker

`Dockerfile` دو مرحله دارد: build با Node 24 و سرو با nginx. `nginx.conf.template` مسیرهای Angular را به `index.html` برمی‌گرداند، فایل‌های hashدار را یک سال کش می‌کند، `index.html` را کش نمی‌کند و `/api` را به `API_URL` پراکسی می‌کند. `compose.yaml` بک‌اند، PostgreSQL و فرانت را کنار هم روی `http://localhost:8080` بالا می‌آورد.

۱۲.۳ Vercel و Render

- فرانت‌اند روی Vercel: preset Angular، دستور `npm run build` ، خروجی `dist/interviewpal/browser` و Node 24.x.
- `vercel.json` مسیر `/api/*` را به `https://interviewpal-backend.onrender.com` می‌فرستد و بقیهٔ مسیرها را به `index.html` ؛ بنابراین مرورگر فقط یک origin می‌بیند و CORS لازم نیست.
- بک‌اند روی Render با Docker اجرا می‌شود. برای حساب کاربری باید دیتابیس PostgreSQL پایدار و متغیرهای `Auth__JwtKey` (حداقل ۳۲ کاراکتر)، `Database__Provider=Postgres` و `ConnectionStrings__Default` تنظیم شود؛ دیسک پلن رایگان Render پایدار نیست و SQLite آن بعد از هر استقرار پاک می‌شود (جزئیات در مستند بک‌اند). در پلن رایگان پس از ۱۵ دقیقه بی‌کاری می‌خوابد و اولین درخواست بعدی حدود نیم دقیقه طول می‌کشد.

۱۲.۴ مستندات

این سند در `docs/frontend.md` نوشته می‌شود و `docs/InterviewPal-Frontend.pdf` از روی آن ساخته می‌شود. با هر تغییر مهم، README، همین سند و PDF با هم به‌روز می‌شوند.

۱۳. محدودیت‌ها و گام‌های بعدی

موضوع	وضعیت فعلی	گام بعدی
تأیید ایمیل و فراموشی رمز	ندارد؛ ایمیل تأیید نمی‌شود و رمز فراموش‌شده بازیابی نمی‌شود	ارسال ایمیل (لینک تأیید و بازنشانی رمز)
ورود اجتماعی	ندارد	ورود با Google و GitHub
حالت مصاحبهٔ زمان‌دار	تایمر ساده سمت کلاینت	محاسبهٔ زمان در سرور برای جلوگیری از دستکاری
مرور فاصله‌دار	فقط فهرست سؤال‌های ضعیف	زمان‌بندی مرور بر اساس منحنی فراموشی
پنل ادمین	محتوا با فایل JSON ویرایش می‌شود	پنل مدیریت سؤال و درس و بررسی گزارش‌ها
مصاحبه‌کنندهٔ هوشمند	ندارد	پاسخ متنی/صوتی و بازخورد
خواب بک‌اند رایگان	تأخیر اولین درخواست	پلن بدون خواب یا پینگ دوره‌ای